Ders 12 Çalışma Özeti

Ders 12 Çalışma Özeti

Genel Konular

- Ders 1 Haziran 2021'de yapıldı.
- Önce kalan makale sunumları tamamlandı, sonra uygulama projesi sunumlarına geçildi.
- Makale sunumları:
  - 1. sunum: profesyonel yazılımcıların kod yazarken yaptığı beceri tabanlı insan hataları ve bu hataları azaltma stratejileri.
  - 2. sunum: DO-178C süreçlerini izleyen güvenlik kritik (safety-critical) bir yazılımda COCOMO II ile efor tahmini.
  - 3. sunum: çevik yöntemin, yani Scrum'ın yazılım proje yönetimine etkisi (anket çalışması).
- Uygulama projesi sunumları:
  - 1. uygulama: sera sıcaklık kontrol sistemi (Arduino ve Android).
  - 2. uygulama: uzaktan kumandalı, kameralı robot kontrol sistemi.
  - 3. uygulama: stok yönetim sistemi (şelale modeli, fonksiyon noktası ve COCOMO tahmini, birim testler).
- Dersin sonunda hoca final sınavı ve vize notları hakkında bilgi verdi.

Hocanın Özellikle Vurguladığı Kısımlar

- Şelale (waterfall) modelinde üst düzey aşamalar bitiş-başlangıç (finish-to-start) ilişkisiyle bağlanır: analiz bitince tasarım, tasarım bitince gerçekleme, gerçekleme bitince test başlar.
- Ama bir aşamanın alt iş adımlarının hepsi bitiş-başlangıç olmak zorunda değildir. Bazıları paralel yürüyebilir ve başlangıç-başlangıç (start-to-start) ilişkisi kurulabilir. Örneğin SRS dokümanının bölümleri paralel yazılabilir.
- "Şelale ise her şey bitiş-başlangıç olmalı" diye bir koşul yoktur. Hoca bunu diğer öğrenciler için de vurguladı.
- Birim test (unit test) asıl olarak yazılım mühendisliği ve yazılım kalite derslerinin konusu. Yine de örnek olarak göstermek faydalı.
- Uygulama projesi için yüklenecekler: proje raporu ve SRS belgesi. İkisi tek dosyada da olabilir.
- Rapor yükleme süresi uzatılmayacak, bu gün sona eriyor. Eksik kalan eklemeler sunum sırasında anlatılabilir.
- Sunumunu zamanında yapıp dosyayı yükleme tarihini kaçıran öğrenci dosyayı hocaya e-postayla gönderebilir.
- Online eğitimde de ders devamı sistemde görülüyor. Devamsızlığı fazla olanlar için bölümün genel kararı uygulanacak.
- 1. makale sunumuna geri bildirim: IEEE Transactions on Software Engineering'den güncel bir dergi makalesi seçmesi güzel. Zamanını iyi kullanamadı. Çalışma bir anketle genişletilebilir.
- 2. makale sunumuna geri bildirim: derste görülen COCOMO'yu havacılık alanında uygulayan bir makale seçmiş, zamanını da iyi kullanmış.

Kısa Tekrar Notları

- Rasmussen'e göre hatalar beceri tabanlı (skill-based), kural tabanlı (rule-based) ve bilgi tabanlı (knowledge-based) olarak ayrılır.
- Büyük olasılıkla Reason'a göre (altyazıda "Erasmus'a göre") yürütme hataları kayma (slip) ve unutma (lapse), planlama hataları ise yanılgıdır (mistake).
- Yazılımcıların bildirdiği en yaygın hata nedenleri: geliştirme ortamının karmaşıklığı ile konsantrasyon ve odaklanma sorunları.
- Hata azaltma stratejileri dört grupta toplanır: geliştirici, süreç, araç, yönetim.
- COCOMO II: efor kişi-ay (person-month, PM) olarak hesaplanır. 5 ölçek faktörü (PREC, FLEX, RESL, TEAM, PMAT) ve 17 efor çarpanı vardır.
- Üs $E > 1$ ise ölçekten kaynaklanan verimsizlik (diseconomy of scale) vardır: proje boyutu iki katına çıkınca efor iki katından fazla artar.
- DO-178C yazılım seviyeleri: A (felaket), B (tehlikeli), C (büyük), D (küçük), E (güvenlik etkisi yok).
- Göreli hata büyüklüğü (Magnitude of Relative Error, MRE), tahmin ile gerçekleşen efor arasındaki farkın gerçekleşen efora oranıdır.
- Uygulama projelerinde risk tablosu: risk, etki, olasılık, risk tipi, B planı, C planı.
- Final sınavı programdaki saatte yapılacak, büyük ihtimalle test usulü.

Detaylı Açıklamalar

Ders başı: teslimler ve devam

- Hoca bu hafta sunacakları tek tek belirledi. Önce makale sunumları, sonra uygulama sunumları yapılacak.
- Hoca tatiller araya girerse yetişilmez diye uygulama raporlarının yükleme tanımını sisteme yapmıştı. Ama anlatmak için zaman kaldığı için sunumlar derste yapılabildi.
- Bugün sunamayanlar önümüzdeki hafta daha kısa sunacak.

1. makale sunumu: yazılımcıların beceri tabanlı hataları

- **Makale:** IEEE Transactions on Software Engineering, 2020-2021.
- **Bağlam:** konu yazılım kaliteyle kesişiyor. Ama hata teknik açıdan değil, yazılım proje yönetimindeki insan (people) bakış açısından ele alındığı için bu derse uygun bulundu.
- **Problem:** yazılımcılar kod yazarken hata yapmaya yatkındır. Şimdiye kadar çoğu çalışma hataların teknik nedenlerine bakmış ve karmaşık araçlar önermiş. Hataları insan açısından inceleyen çalışma az.
- **Örnek yazılım hataları:** bir kripto para borsasında yüz binlerce bitcoinin kaybı, OpenSSL'deki Heartbleed açığı, Mars Climate Orbiter'ın ölçü birimi uyuşmazlığı yüzünden kaybı.
- **Önceki çalışmalar:** depo madenciliği (repository mining), duygu analizi ve doğal dil işleme ile yazılımcının ruh hâlinin kod düzeltmelerine etkisi incelenmiş. Örneğin üzgün yazılımcıların hataları düzeltmesi mutlu olanlara göre daha uzun sürüyor.
- **Hata nedenleri:** psikolojik (bölünme, dikkat dağınıklığı, öfke, hayal kırıklığı), fiziksel (yorgunluk, uykusuzluk), çevresel (sıcaklık, soğuk).
- **İnsan hatası teorisi:**
  - Rasmussen'in çerçevesi: beceri tabanlı hatalar iyi bilinen görevlerdeki otomatik davranışlarda, kural tabanlı hatalar kayıtlı kurallarla kontrol edilen durumlarda, bilgi tabanlı hatalar bilinçli akıl yürütme gerektiren beklenmedik durumlarda olur.
  - Çalışma beceri tabanlı hatalara odaklanmış. Bu hataları fark etmek ve düzeltmek daha kolay.
  - Yürütme hataları kayma (slip, örneğin yanlış tuşa basmak) ve unutmadır (lapse, örneğin yapılacak işi unutmak). Planlama hataları yanılgıdır (mistake) ve kural ya da bilgi tabanlıdır.
- **Sekiz beceri tabanlı hata türü** (gerçek hayat örnekleriyle):
  - İhmal (omission): bir adımın atlanması. Örnek: çaydanlığın altını yakmayı unutmak.
  - Tekrar (repetition): tamamlanmış bir adımın yeniden yapılması.
  - Tersine çevirme (reversal): araya giren bir işten sonra yanlış adıma dönmek. Örnek: terlik giyecekken düşen ceketi kaldırıp ayakkabıyı yeniden giymek.

  - Bölünme sonrası ihmal: dış bir neden yüzünden bir şeyi unutmak. Örnek: ceketi alıp telefona cevap verdikten sonra ceketsiz çıkmak.
  - Çifte yakalama kayması (double capture slip): ilişkili başka bir işin istemeden yapılması. Örnek: ayakkabıyı çıkarırken çorabı da çıkarmak.
  - Azalmış niyet (reduced intentionality): gecikme yüzünden ne yapılacağının unutulması.
  - Algısal karışıklık (perceptual confusion): alışkanlıkla yanlış nesneyi seçmek. Örnek: her zaman süt şişesi alındığı için su yerine süt şişesini almak.
  - Girişim hatası (interference error): iki aktif planın birbirine karışması. Örnek: telefonu kapatırken içeri girene "alo, kimsiniz" demek.
- **Yöntem:**
  - Twitter duyurusu ve kulaktan kulağa yolla endüstride çalışan 27 yazılımcı bulunmuş.
  - Skype ya da yüz yüze 30-45 dakikalık mülakatlar yapılmış. Önce sekiz hata türü örneklerle açıklanmış, sonra katılımcılara bu hataları yapıp yapmadıkları ve nasıl azalttıkları sorulmuş.
  - Kayıtlar yazıya dökülmüş, ayrı ayrı kodlanıp karşılaştırılmış. 57 farklı hata teması belirlenmiş.
- **Bulgular:**
  - En sık bildirilen beceri tabanlı hatalar: ihmal ve bölünme sonrası ihmal.
  - En sık neden: geliştirme ortamının karmaşıklığı (birden fazla dil, araç ve bağımlılık). Örnek: bir betiği hedef veritabanı yerine başka bir veritabanında çalıştırmak.
  - Konsantrasyon bozukluğunun nedenleri: görev değişimi, dikkat dağınıklığı, iş baskısı. Örnek: e-postaya cevap verdikten sonra ne yapıldığını unutmak.
  - Strateji grupları: geliştirici, süreç, araç, yönetim.
  - Yazılımcılar çoğunlukla kendi irade ve disiplinlerine dayanan stratejiler kullanıyor. Ama planlı destek olmadan irade tek başına yetmiyor.
  - Kulaklık kullanmak ve bildirimleri kapatmak bölünmeleri azaltıyor.
  - Süreç stratejileri (kontrol listesi, kod gözden geçirme, test) hatayı önlemekten çok hızlı tespit edip düzeltmeye yarıyor.
  - Araçlar (örneğin git) yardımcı oluyor.
  - Yönetim stratejileri az anılmış. Yönetimin iş yükünü iyi dağıtması, ödüllendirmesi ve eğitim vermesi gerekiyor.
  - Şaşırtıcı sonuç: dinlenmek, ara vermek hata azaltma stratejisi olarak pek anılmamış. Yorgunluk fark edildiğinde basitçe ara verilerek çözülüyor.
- **İsviçre peyniri modeli (Swiss cheese model):** sistem merkezli bir yaklaşım. Yazılımcının yaptığı bir hata, pull request ve kod gözden geçirme gibi savunma katmanlarıyla canlıya çıkmadan durdurulabilir.
- **Öneriler:**
  - Durum farkındalığını (situational awareness) geliştirmek: küçük sanılan bir sorun büyüyüp asıl görevden uzaklaştırabilir. Bunun için irade dışında destek yaklaşımları gerekir.
  - Bilişsel yetenekleri (odaklanma, hatırlama, disiplin, dikkat) eğitimlerle geliştirmek.
  - Her şeyi hatırlamaya çalışmak yerine kontrol listesi kullanmak.
  - Hızlı geri bildirim döngüleri: eşli programlama (pair programming) ve pull request.
- **Hocanın ek açıklaması:** hoca daha önce öğrencilerle insan faktörü üzerine anket çalışmaları yaptığını söyledi. İş hayatındaki öğrenciler benzer bir anketle bu çalışmayı geliştirebilir.

2. makale sunumu: DO-178C güvenlik kritik yazılımda COCOMO II

- **Makale:** Journal of Aerospace Technology and Management, Brezilya, 2019.
- **Güvenlik kritik yazılım:**
  - İnsan güvenliği açısından kritik donanım ve yazılımı kontrol eden yazılımdır. Genellikle gömülü ve gerçek zamanlıdır. Tek bir arıza felakete yol açabilir.
  - Standartlar: havacılıkta DO-178C, uzayda NASA-STD-8719.13C, tıpta IEC 62304.
  - ABD'de havacılık otoritesi FAA, Avrupa'da EASA. DO standartlarının Avrupa'daki karşılıkları ED belgeleridir.
  - DO-178C bir rehber dokümandır, ama otoriteler ekipman spesifikasyonlarında ona atıf yaparak uyulmasını ister.
  - Geçmişte uçak geliştirmede en büyük zorluk aerodinamikti, şimdi yazılım.

- **DO-178C seviyeleri** (arızanın uçak ve mürettebata etkisine göre, uçuş saati başına olasılık):
  - A, felaket (catastrophic): 10^{-9}.
  - B, tehlikeli (hazardous): 10^{-7}.
  - C, büyük (major): 10^{-5}.
  - D, küçük (minor): 10^{-3}.
  - E: güvenlik etkisi yok.
  - Seviye, sistem güvenlik değerlendirme süreciyle (ARP 4761, hata ağacı analizi, fonksiyonel tehlike analizi) belirlenir.
  - Seviye yükseldikçe karşılanması gereken hedef (objective) sayısı artar: A 71, B 69, C 62.
- **Efor tahmin yöntemleri:**
  - Algoritmik olmayan: uzman görüşü, analoji, karşılaştırma. Matematiksel denklem kullanılmaz.
  - Algoritmik: genelde kod satır sayısından matematiksel denklemlerle efor hesaplanır. COCOMO II bu gruptadır.
- **COCOMO II (Constructive Cost Model):**
  - İlk sürümü 1981'de, COCOMO II 2000'de yayımlandı.
  - Efor kişi-ay (PM) cinsinden; girdiler sabitler, kod boyutu (bin satır), efor çarpanları ve ölçek faktörleri.
  - Geliştirme süresi (Time to Develop, TDEV) de hesaplanır.
  - 5 ölçek faktörü: PREC (önceki projelere benzerlik), FLEX (geliştirme esnekliği), RESL (risk çözümleme), TEAM (ekip uyumu), PMAT (CMMI'ya göre süreç olgunluğu). Her biri çok düşükten çok yükseğe altı seviyede.
  - 17 efor çarpanı dört grupta: ürün (güvenilirlik, veritabanı boyutu, karmaşıklık, yeniden kullanılabilirlik, dokümantasyon), donanım/platform (yürütme süresi kısıtı, depolama kısıtı, platform oynaklığı), personel (analist ve programcı yeteneği, personel sürekliliği, deneyimler), proje (araç kullanımı, çok konumlu geliştirme, takvim).
- **Vaka:**
  - Brezilya'da 20 yıldır uçuş kontrol (fly-by-wire) yazılımı geliştiren bir şirketin A seviyesi projesi. Temmuz 2013-Aralık 2017. Aynı anda en fazla 48 yazılım mühendisi. Basit donanım arayüzleri için bir alt yüklenici kullanılmış.
  - Uzman görüşü ve benzetmeyle 140 bin satır kod tahmin edilmiş.
  - Ölçek faktörleri: benzerlik nominal, esneklik çok düşük, risk çözümleme nominal, ekip çok düşük, olgunluk çok yüksek. Sonuçta $E > 1$.
  - Örnek çarpanlar: güvenilirlik çok yüksek, karmaşıklık çok yüksek, dokümantasyon ihtiyacı çok yüksek, araç kullanımı çok düşük.
  - Tahmin: 1257 kişi-ay (1 ay = 152 saat). 41 tam zamanlı personelle 30,6 ayda bitmesi bekleniyordu.
  - Proje V modeliyle yürütüldü. DO-178C'de entegrasyon ve doğrulama süreçlerinin ağırlığı COCOMO II'nin varsaydığı şelaleye göre daha fazla.
  - Gerçekleşen: 1780 kişi-ay, 54 ay, ortalama yaklaşık 32,9 kişi, 170 bin satır kod. Aşamalardaki kişi sayıları planlamada 7'den doğrulamada 55'e çıkmış.
  - Sunumda MRE yaklaşık 0,4 (%40 hata) olarak verildi.
- **Sonuç:** COCOMO II, DO-178C'yi izleyen A seviyesi bir yazılımda yetersiz kalmış. Gereksinim gözden geçirme, kod inceleme, test sonuçlarını gözden geçirme ve test çok zaman alıyor. Doğrulama (verification) gibi bazı faaliyetleri geliştiriciden bağımsız kişilerin yapması zorunlu.

3. makale sunumu: Scrum'ın yazılım proje yönetimine etkisi

- **Makale:** IEEE, 2019. Sunucu birkaç destekleyici kaynak da kullandı. Konu 10. dersteki 5. sunumla aynı anket çalışmasına dayanıyor.
- Yazılım projelerinin başarısızlık nedenleri: planlama eksikliği, proje yönetimi tekniklerinin ve araçlarının eksikliği.
- Örnek felaketler: kanser tedavisinde kullanılan bir cihazın yazılım hatası yüzünden hastaların ölümü, Ariane 5 roketinin kaybı, Almanya'da banka kartlarının çalışmaması.
- PMBOK'un 10 bilgi alanı: entegrasyon, kapsam, zaman, maliyet, kalite, insan kaynakları, iletişim, risk, tedarik, paydaş yönetimi.

- Başarılı proje: iş gereksinimlerini karşılar, zamanında ve bütçe içinde teslim edilir, gerekli destek sağlanır.
- Scrum: üç rol (ürün sahibi, geliştirme ekibi, Scrum Master). Aşamalar başlatma, planlama, uygulama, kontrol, yayınlama. Sprintler 3-4 hafta. Sunucuya göre sektörün %65'i Scrum'a geçmiş.
- **Anket** (30'dan fazla yazılım şirketi):
  - Kapsam yönetiminde olumlu etki var. Müşteriyle sık toplantılar değişikliklere hızlı yanıt sağlıyor.
  - Maliyet yönetiminde yaklaşık %95,2 olumlu.
  - Kalite yönetiminde %15 kesinlikle katılıyorum, %55 katılıyorum.
  - Risk yönetiminde yaklaşık %14 olumlu, %66 "her zaman bir miktar risk vardır".
  - İnsan kaynakları yönetiminde altyazıda "yüzde yirmi beş, yüzde elli olumlu" diye geçiyor.
- **Sonuç:** Scrum süre, maliyet, kapsam, kalite ve risk üzerinde olumlu etkili.
- Hoca Scrum'ın temel kavramlarının bilindiğini söyleyerek bu kısmın hızlı geçilmesini istedi.

1. uygulama: sera sıcaklık kontrol sistemi

- **Konu:** iki öğrencinin hazırladığı basit bir sera sıcaklık kontrol sisteminin proje planı.
- **Sistem:** Arduino Uno, LM35 sıcaklık sensörü, Ethernet Shield, MySQL veritabanı, fan, ısıtıcı, 16x2 LCD ekran, bir IoT platformu ve Android uygulama.
- **Use case'ler:** kullanıcı girişi, profil güncelleme, sera yönetimi (fan ve ısıtıcıyı açıp kapatma), anlık sıcaklığı görüntüleme, saatlik verilerden günlük rapor (çubuk ve çizgi grafik).
- **Yöntem:** Scrum. Ürün sahibi, geliştirme ekibi ve Scrum Master rolleri tanımlanmış. Dört sprint, her biri iki hafta. Her sprint testle bitiyor.
- **Risk analizi** (etki, olasılık, tip, B planı, C planı). Risk tipleri: zaman (Z), kullanıcı (K), teknik (T), personel (P).
  - Eğitim ve deneyim eksikliği: B planı eğitim, C planı sürenin uzatılması.
  - Arayüzün verimli kullanılamaması: B planı arayüzü kullanıcı dostu yapmak, C planı arayüz çatısını değiştirmek.
  - Arduino ve Ethernet Shield'ın bağlanamaması, veritabanı bağlantı sorunu, sensörün yanlış ölçümü, ekranın yanlış göstermesi: yedek malzeme ve uzman desteği, olmazsa cihaz, marka ya da sistem değişikliği.
  - Pandemi nedeniyle personelin hastalanması: bilgi yedekliliği; Scrum'da tüm ekip işten haberdar olduğu için iş sürebilir.
- **Zaman planı:** iş kırılım yapısı (Work Breakdown Structure, WBS) ve Gantt şeması. Görevler arası bağımlılıklar ve paralel yürüyen işler gösterilmiş.
- **Kalite ölçüleri:** kapsama uygunluk (SRS'e göre), zaman planına uygunluk, bütçeye uygunluk, seçilen geliştirme modeline uygunluk (ürün sahibi denetler).
- **Hocanın değerlendirmesi:** seçilen konuda bir proje planının özeti gibi hazırlanmış.

2. uygulama: robot kontrol sistemi

- **Konu:** çevreden görüntü ve video alan bir robotun Bluetooth ile mobil ve masaüstü uygulamalardan uzaktan kontrolü. İki öğrenci sundu.
- **Kapsam ve kısıtlar:**
  - Robot yaklaşık 50 cm boyunda, yürüyebiliyor.
  - Kayıt e-posta, Facebook ya da Twitter hesabıyla, doğrulamalı.
  - Bluetooth menzili yaklaşık 25 metre.
  - Dört yönde hareket ve zıplama.
  - En fazla 100 MB video, 720p kayıt.
  - Son bir aylık kullanım verileri.
- **Yöntem:** çevik ve Scrum, 21 günlük (3 haftalık) sprintler. Her sprint sonunda tüm ekip ve yöneticilerin katıldığı değerlendirme toplantısı.
- **Bütçe:** donanım yaklaşık 100.000 TL, personel 588.000 TL, pazarlama 200.000 TL.
- **Ekip:** analiz, backend, frontend, haberleşme ve test ekipleri (kıdemli ve yeni başlayan personel) ile bir ürün sahibi.

- **Planlama:** MS Project ile 3 seviyeli bir WBS. Örnek olarak planlama ve analiz aşamaları gösterildi.
- **Riskler** (altı başlık): plan gecikmesi, geliştirme ortamının kurulamaması, ekipten birinin ayrılması, ekibin yetkinliğinin yetersizliği, karışık ve açıklamasız kod (önlem: kodlama standartları, eşli programlama), yetersiz test (önlem: teste yeterli efor ve süre ayırmak).
- **Arayüz:** giriş/kayıt ekranı, yön tuşları ve zıplama butonuyla kontrol ekranı, kamera görüntüsü, video ve fotoğraf kaydı, pil, bakım ve sinyal bilgilerini ve aktiflik grafiğini gösteren rapor ekranı.
- **Fonksiyonel olmayan gereksinimler:** anlaşılır ve göz yormayan arayüz, anlaşılır hata mesajları, çoklu platform desteği, düşük yanıt süresi, kişisel verilerin güvenli saklanması.
- **Araçlar:** Jira, NetBeans, Oracle SQL Developer, Trello.
- **Hocanın değerlendirmesi:** derste belirtilen proje yönetim belgeleri hazırlanmış.

3. uygulama: stok yönetim sistemi

- **Konu:** küçük ve orta ölçekli işletmeler için hazır bulunan bir stok yönetim yazılımının proje planı ve testleri. Ürün, kategori ve kullanıcı yönetimi ile raporlar.
- **Yöntem:** şelale modeli. Her aşamada müşteri onayı alınıyor: başlama planı, SRS, SPMP (yazılım proje yönetim planı), yazılım tasarım dokümanı, kodlama, test, teslim.
- **Ekip:** 5 kişi (proje yöneticisi, tasarımcı, backend, frontend, test).
- **Maliyet tahmini:**
  - Fonksiyon noktası (function point): girdi, çıktı, sorgu, dosya ve dış arayüz sayıları ile değer ayarlama soruları.
  - PHP katsayısıyla satır sayısına dönüştürme, ardından COCOMO.
  - Sonuç: yaklaşık 7,8 ay ve yaklaşık 30 bin dolar.
- **Araçlar:** Linux kullandığı için MS Project yerine ProjectLibre, Photoshop yerine GIMP. PHP, HTML5, CSS, JavaScript, MySQL, LibreOffice ve DataGrip.
- **Planlar:** proje kontrol planı (sabit takvim), kalite kontrol planı (her kilometre taşında test ve müşteri onayı), risk tablosu, kabul planı (belgelerin zamanında ve IEEE standartlarına uygun teslimi).
- **Testler:**
  - Kullanıcı girişi, kullanıcı kaydı, ürün ekleme ve kategori ekleme için test senaryoları yazılmış.
  - PHPUnit ile 3 modül için 3 birim test sınıfı ve toplam 38 test çalıştırılmış, hepsi geçmiş.
  - Sunucu derste beklenen değerle farklı bir değer vererek ve kodu bilerek bozarak testin nasıl başarısız olduğunu gösterdi. Bir yeri değiştirirken başka bir yeri bozmayı yakalamak için faydalı.
- **Hocanın değerlendirmesi:**
  - Test derslerinde genelde Java ve JUnit kullanılıyordu, PHPUnit örneği görmek iyi oldu.
  - ProjectLibre'deki planda tüm alt adımların bitiş-başlangıç bağlanması yanlış bir genelleme (yukarıdaki vurguya bakın).

Kapanış: final ve vize

- Final sınavı ders programında belirtilen saatte yapılacak. Büyük ihtimalle test usulü olacak.
- Vize kâğıtları okundu, notlar yaklaşık 68 ile 100 arasında. Not sisteminde vize notu için yer olmadığından hoca notları kısa bir süre için online sistemdeki ders duvarında duyurabileceğini söyledi.

Ders sonrası notlar

- 1. makale sunumunda "Mutibox" diye geçen olay Mt. Gox borsasıdır. Borsa 2014'te yaklaşık 850 bin bitcoin kaybını açıklayarak çöktü; altyazıdaki 2011 yılı saldırıların başladığı döneme denk gelir.
- Heartbleed, OpenSSL kütüphanesinde 2014'te ortaya çıkan bir açıktır.
- Mars Climate Orbiter (1999), bir ekibin İngiliz ölçü birimlerini (pound-force), diğer ekibin metrik birimleri kullanması yüzünden kaybedildi. İki ekip de ABD'deydi (Lockheed Martin

ve NASA JPL). Sunumda “Avrupa ve Amerika” farkı denmesi yanlıştır.

- 2. makale sunumunda aynı anda en fazla 48 yazılım mühendisi çalıştığı söylendi, ama entegrasyon için 50, doğrulama için 55 kişi okundu. Bu iki bilgi çelişiyor. Altyazıdan çelişkinin nedeni anlaşılmıyor.
- Altyazıda “Erasmus” ve “Ziple Lep” diye geçen adlar büyük olasılıkla James Reason ve onun “slips and lapses” (kaymalar ve unutmalar) sınıflamasıdır. İsviçre peyniri modeli de büyük olasılıkla Reason’a aittir.
- Altyazıda “bilgisayar yetenekleri” diye geçen ifade bağlama göre “bilişsel (cognitive) yetenekler” olmalıdır.
- 2. makale sunumunda hoca DO’nun açılımını sordu. Sunucu “Design Organization” dedi; bu doğru değil. DO, standardı yayımlayan RTCA’nın belge numarası önekidir (“RTCA DO-178C”). Avrupa karşılığı EUROCAE ED-12C’dir.
- DO-178C’de D seviyesi için hedef sayısı sunumda 24 olarak verildi. Standarttaki değer 26’dır.
- Sunumda verilen sayılarla MRE hesaplanırsa: $|1780 - 1257|/1780 \approx 0,29$. Sunumda verilen 0,4 değeri, fark tahmini efora bölündüğünde çıkan değere ($523/1257 \approx 0,42$) daha yakın. MRE’nin standart tanımında payda gerçekleşen efordur.
- 3. makale sunumunda kapsam yönetimi için verilen oran altyazıda anlaşılmıyor.
- Altyazıda “Airline 5” diye geçen olay Ariane 5 roketinin 1996’daki ilk uçuşunda yazılım hatası yüzünden kaybıdır.
- 1. uygulama sunumunda sprintler arası ilişki “start-to-finish” diye anlatıldı. Bir sprint bitince diğerinin başladığı anlatıldığına göre doğru terim finish-to-start’tır.
- 3. uygulama sunumundaki “IT-FIL standartları” IEEE standartları olmalıdır. Tasarım aracı “DataGrid” ise JetBrains’in DataGrip aracıdır.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.